



Robot PuppyPi

Guía de Uso con Python

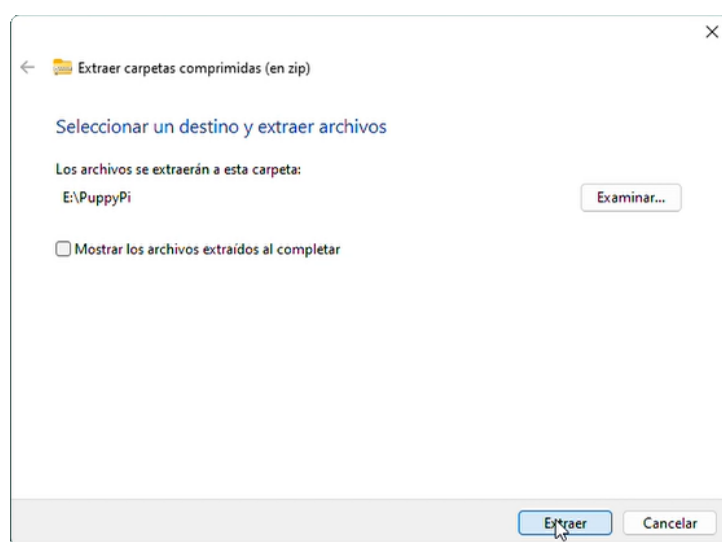
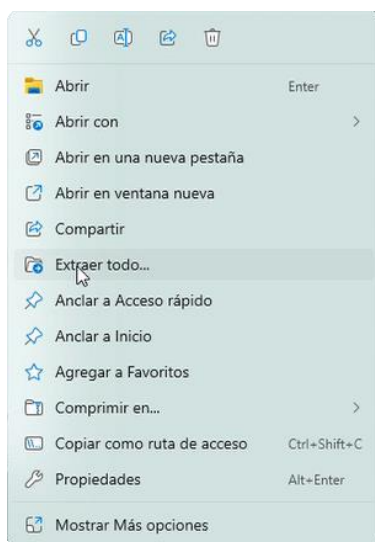
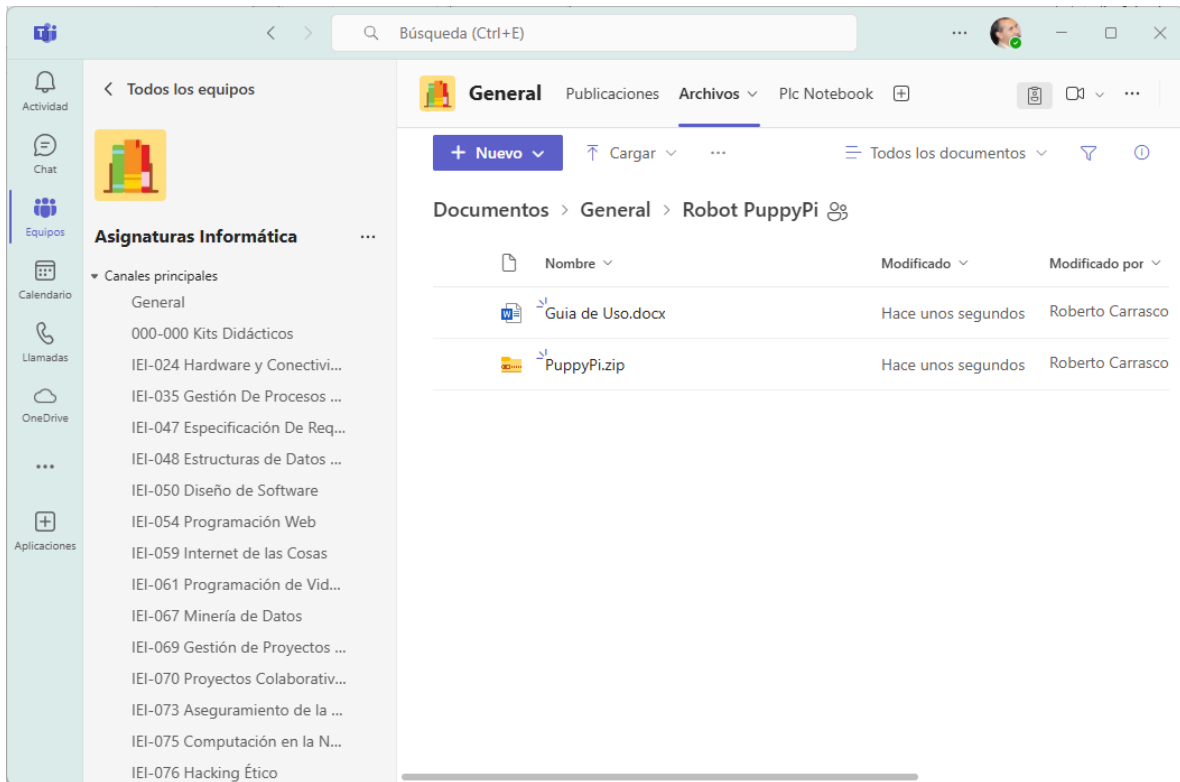
2025

Contenido

Instalación del código	3
Creación del entorno virtual de Python con VSCode	4
Instalación de los módulos requeridos	8
Demo modo gráfico	9
Demo modo texto	10

Instalación del código

Descargue el archivo **PuppyPi.zip** desde Teams y descomprímalo en su computador



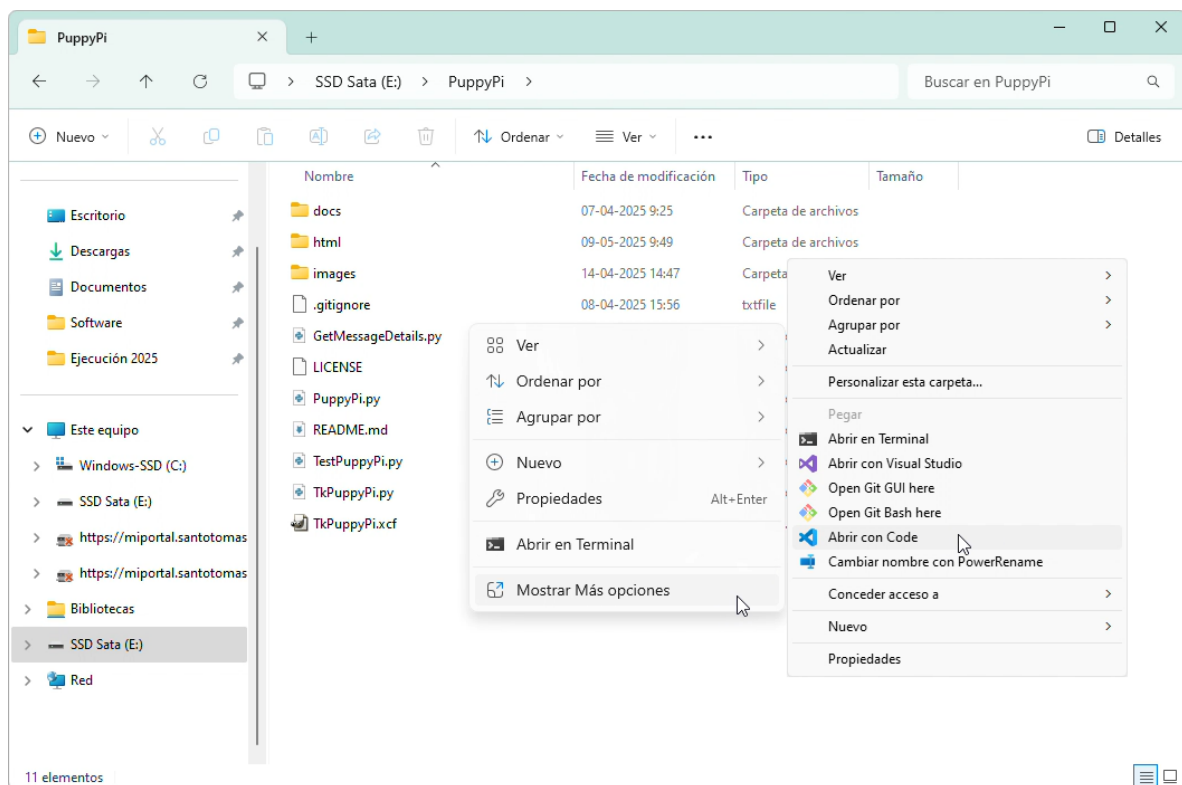
Creación del entorno virtual de Python con VSCode

Las aplicaciones en Python usualmente hacen uso de paquetes y módulos que no forman parte de la librería estándar. Las aplicaciones a veces necesitan una versión específica de una librería, debido a que dicha aplicación requiere que un bug particular haya sido solucionado o bien la aplicación ha sido escrita usando una versión obsoleta de la interfaz de la librería.

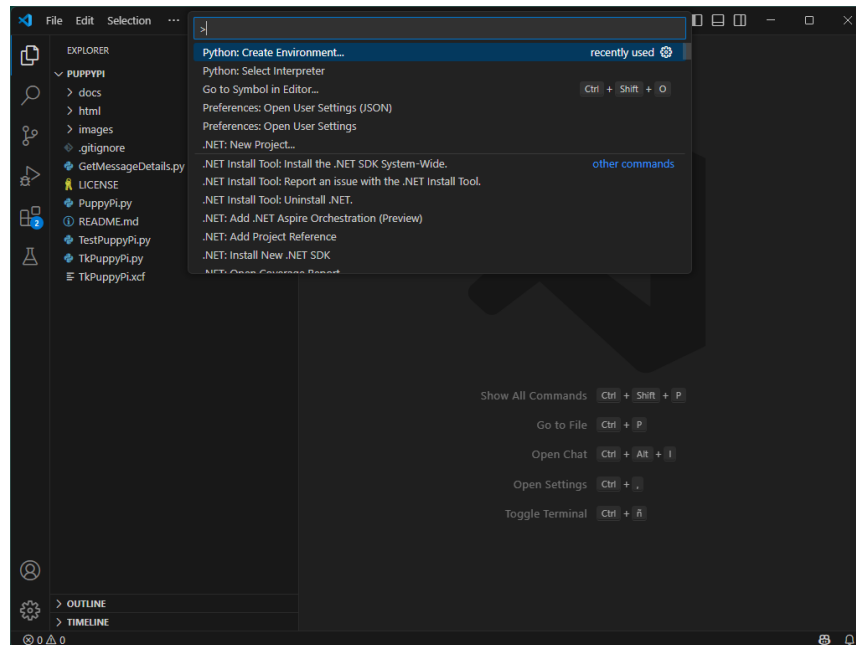
Esto significa que tal vez no sea posible para una instalación de Python cumplir los requerimientos de todas las aplicaciones. Si la aplicación A necesita la versión 1.0 de un módulo particular y la aplicación B necesita la versión 2.0, entonces los requerimientos entran en conflicto e instalar la versión 1.0 o 2.0 dejará una de las aplicaciones sin funcionar.

La solución a este problema es crear un entorno virtual, un directorio que contiene una instalación de Python de una versión en particular, además de unos cuantos paquetes adicionales.

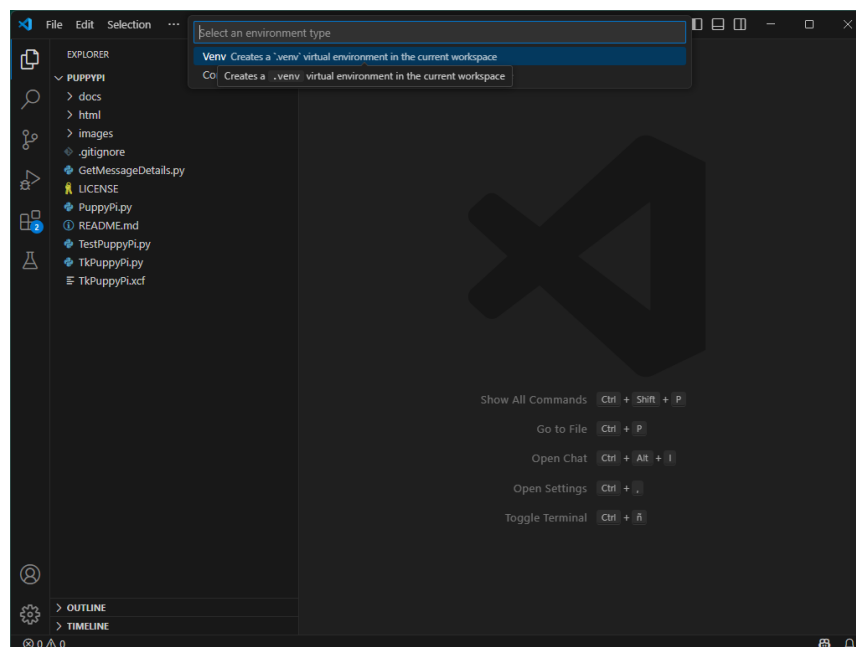
Vaya a la carpeta de **PuppyPi**, haga click con el botón derecho del mouse, seleccione “**Mostrar Más opciones**” y finalmente haga clic en “**Abrir con Code**” (Visual Studio Code – VSCode)



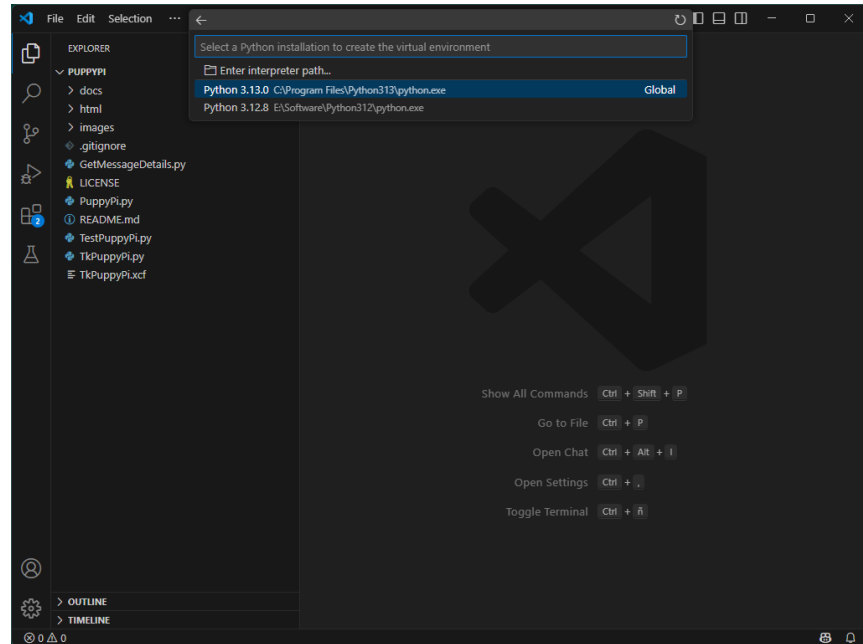
En VSCode presione simultáneamente las teclas **SHIFT-CTRL-P** para acceder a la **Paleta de Comandos** y luego seleccione la opción **“Python: Create Environment...”**



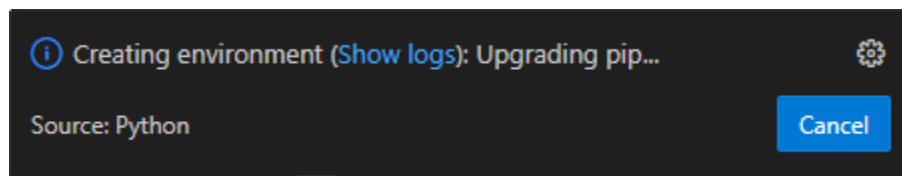
Luego seleccione el tipo de entorno virtual a crear, para nuestro caso se utilizará Venv, **“Venv Creates a `.venv` virtual environment in the current workspace”**



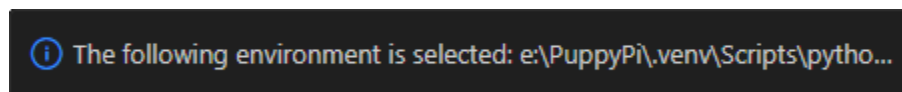
Ahora debe seleccionar el intérprete de Python que utilizará dentro de este entorno virtual, en este caso el que aparece como **“Global”** y que corresponde al que está por defecto en su sistema.



En la parte inferior derecha de la ventana aparecerá un mensaje indicando que el entorno virtual se está instalando

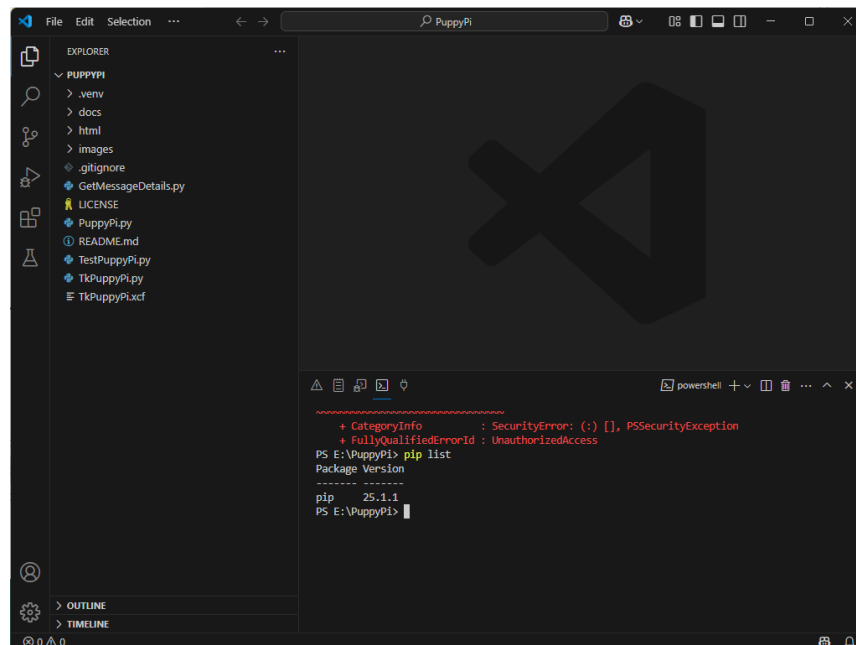
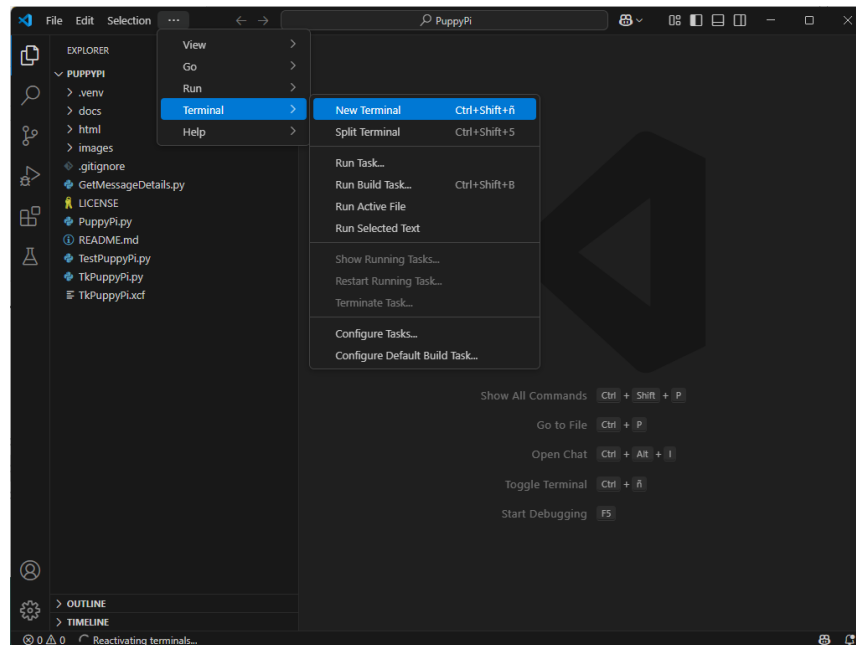


Y luego un mensaje indicando que la operación de creación del entorno virtual ha finalizado



Hay veces en las cuales VSCode crea el entorno virtual pero no lo activa de manera correcta por lo que es mejor salir de esta aplicación y volver a abrirla, con la carpeta de PuppyPi, tal como se indicó en los pasos previos.

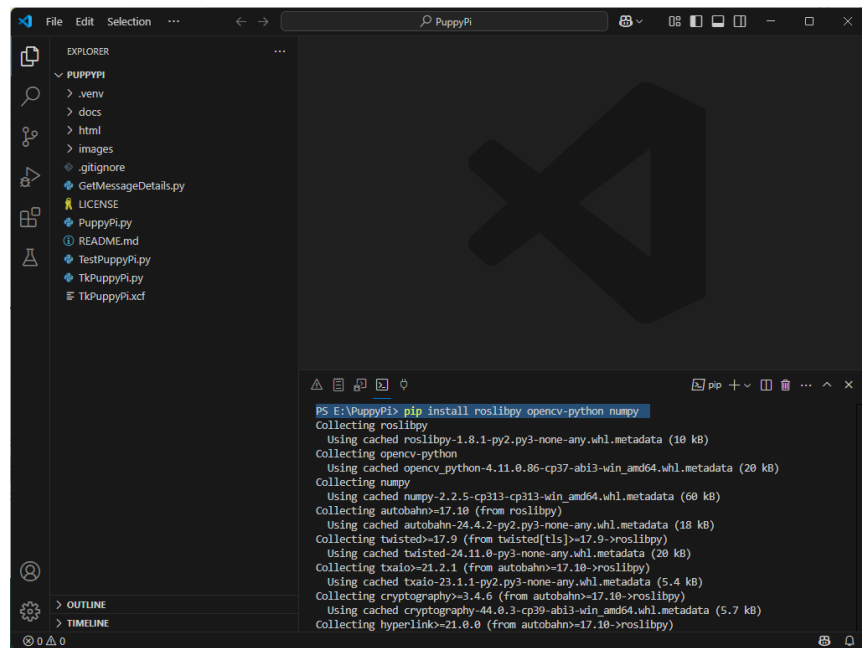
Finalmente debe verificar que el entorno virtual está correctamente instalado para lo cual abra una ventana de terminal y en ella ejecute el comando “**pip list**”



El comando “**pip list**” muestra todos los paquetes (módulos, librerías) instalados en este entorno virtual y en su etapa inicial sólo debería mostrar el módulo “**pip**”

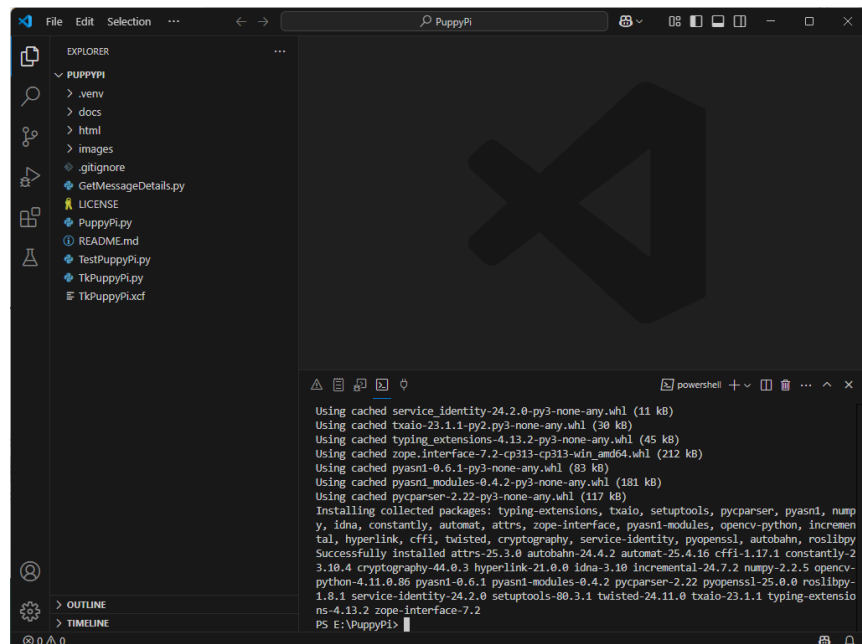
Instalación de los módulos requeridos

Para instalar los módulos requerido por PuppyPi abra una ventana de terminal y ejecute el comando **“pip install roslibpy opencv-python numpy”** lo que instalará los módulos especificados y todas sus dependencias.



The screenshot shows a VS Code editor window with the file explorer on the left displaying the project structure for 'PuppyPi'. The main editor area shows a terminal window with the command `PS E:\PuppyPi> pip install roslibpy opencv-python numpy` and its output. The output lists the collection and installation of several packages and their dependencies, including roslibpy, opencv-python, numpy, autobahn, twisted, txao, cryptography, and hyperlink.

```
PS E:\PuppyPi> pip install roslibpy opencv-python numpy
Collecting roslibpy
  Using cached roslibpy-1.8.1-py2.py3-none-any.whl.metadata (10 kB)
Collecting opencv-python
  Using cached opencv_python-4.11.0.86-cp37-abi3-win_amd64.whl.metadata (20 kB)
Collecting numpy
  Using cached numpy-2.2.5-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting autobahn>=17.10 (from roslibpy)
  Using cached autobahn-24.4.2-py2.py3-none-any.whl.metadata (18 kB)
Collecting twisted>=17.9 (from twisted[tls]>=17.9->roslibpy)
  Using cached twisted-24.11.0-py3-none-any.whl.metadata (20 kB)
Collecting txao>=21.2.1 (from autobahn>=17.10->roslibpy)
  Using cached txao-23.1.1-py2.py3-none-any.whl.metadata (5.4 kB)
Collecting cryptography>=3.4.6 (from autobahn>=17.10->roslibpy)
  Using cached cryptography-44.0.3-cp39-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting hyperlink>=21.0.0 (from autobahn>=17.10->roslibpy)
```

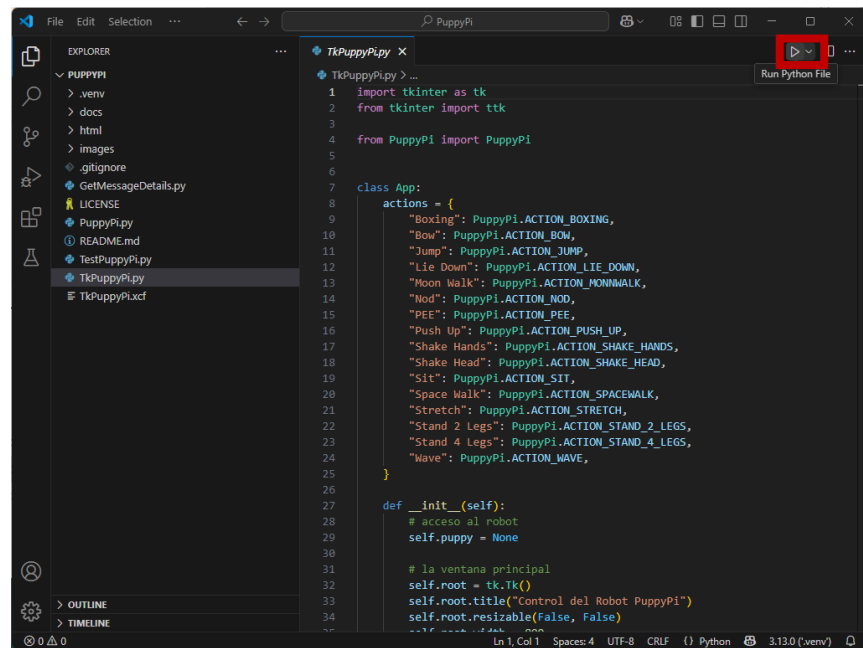


The screenshot shows the same VS Code editor window, but the terminal window now displays the completion of the installation. It lists the packages that were successfully installed, including service_identity, txao, typing_extensions, zope.interface, pyasn1, pyasn1-modules, pycparser, pyasn1, numpy, y, idna, constantly, automat, attrs, zope.interface, pyasn1-modules, opencv-python, incremental, hyperlink, cffi, twisted, cryptography, service_identity, pyopenssl, autobahn, roslibpy, and many others. The terminal prompt is now `PS E:\PuppyPi>`.

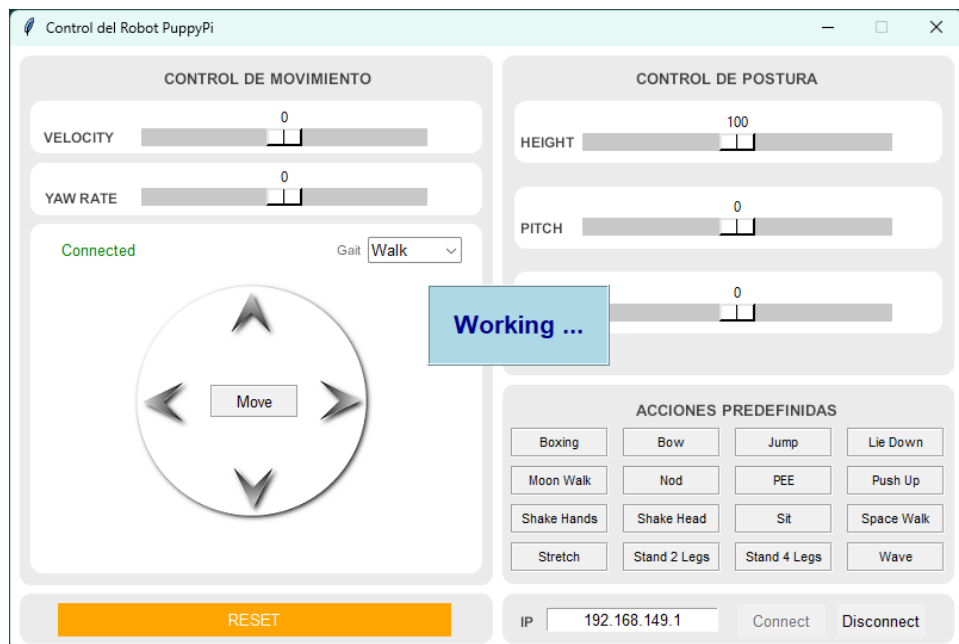
```
Using cached service_identity-24.2.0-py3-none-any.whl (11 kB)
Using cached txao-23.1.1-py2.py3-none-any.whl (30 kB)
Using cached typing_extensions-4.13.2-py3-none-any.whl (45 kB)
Using cached zope.interface-7.2-cp313-cp313-win_amd64.whl (212 kB)
Using cached pyasn1-0.6.1-py3-none-any.whl (83 kB)
Using cached pyasn1-modules-0.4.2-py3-none-any.whl (161 kB)
Using cached pycparser-2.22-py3-none-any.whl (117 kB)
Installing collected packages: typing-extensions, txao, setuptools, pycparser, pyasn1, numpy, y, idna, constantly, automat, attrs, zope.interface, pyasn1-modules, opencv-python, incremental, hyperlink, cffi, twisted, cryptography, service_identity, pyopenssl, autobahn, roslibpy
Successfully installed attrs-25.3.0 autobahn-24.4.2 automat-25.4.16 cffi-1.17.1 constantly-23.10.4 cryptography-44.0.3 hyperlink-21.0.0 idna-3.10 incremental-24.7.2 numpy-2.2.5 opencv-python-4.11.0.86 pyasn1-0.6.1 pyasn1-modules-0.4.2 pycparser-2.22 pyopenssl-25.0.0 roslibpy-1.8.1 service_identity-24.2.0 setuptools-80.3.1 twisted-24.11.0 txao-23.1.1 typing-extensions-4.13.2 zope.interface-7.2
PS E:\PuppyPi>
```

Demo modo gráfico

Para probar el demo en modo gráfico (TkInter) encienda el robot PuppyPi, conecte su computador a la red wifi que el robot crea, cargue y el ejecute el archivo “**TkPuppyPi.py**”, ingrese la IP del robot y presiones el botón “**Connect**”.

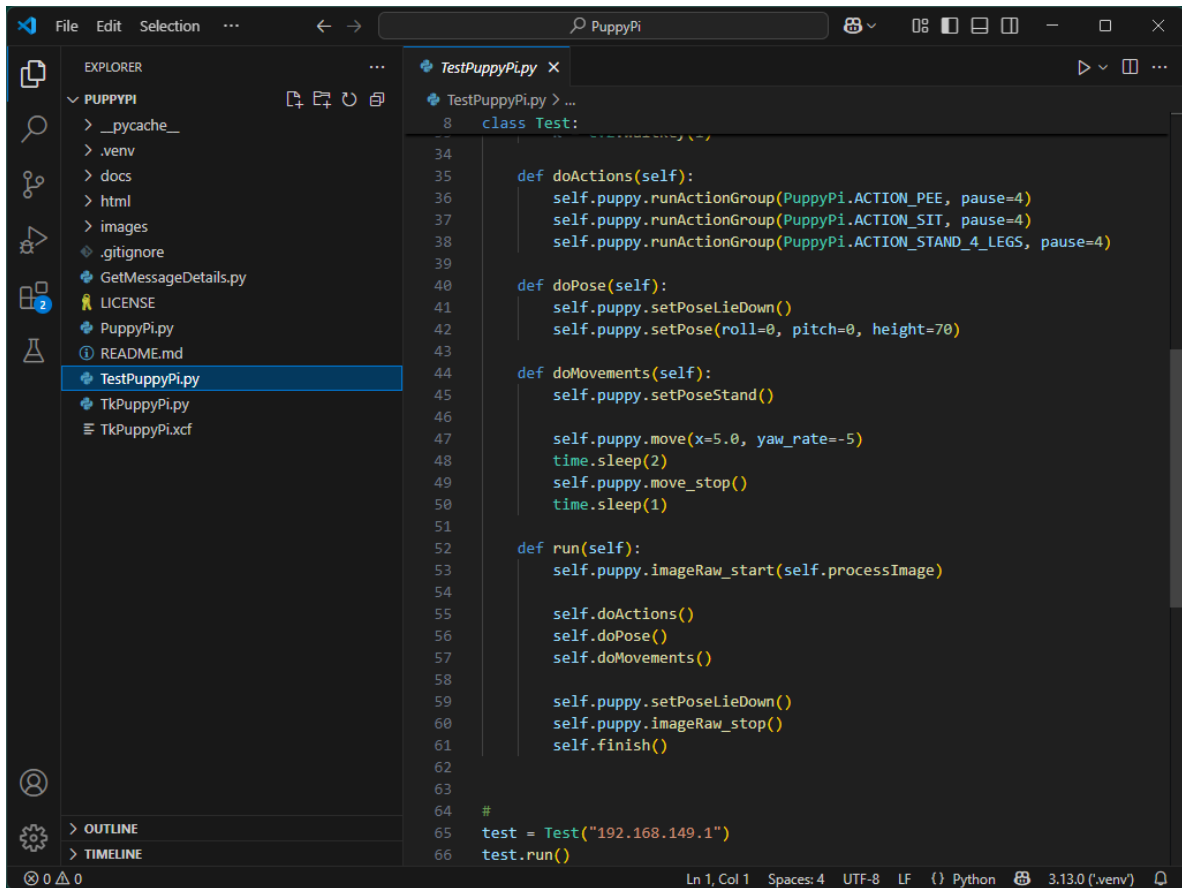


```
1 import tkinter as tk
2 from tkinter import ttk
3
4 from PuppyPi import PuppyPi
5
6
7 class App:
8     actions = {
9         "Boxing": PuppyPi.ACTION_BOXING,
10        "Bow": PuppyPi.ACTION_BOW,
11        "Jump": PuppyPi.ACTION_JUMP,
12        "Lie Down": PuppyPi.ACTION_LIE_DOWN,
13        "Moon Walk": PuppyPi.ACTION_MONNALK,
14        "Nod": PuppyPi.ACTION_NOD,
15        "PEE": PuppyPi.ACTION_PEE,
16        "Push Up": PuppyPi.ACTION_PUSH_UP,
17        "Shake Hands": PuppyPi.ACTION_SHAKE_HANDS,
18        "Shake Head": PuppyPi.ACTION_SHAKE_HEAD,
19        "Sit": PuppyPi.ACTION_SIT,
20        "Space Walk": PuppyPi.ACTION_SPACEWALK,
21        "Stretch": PuppyPi.ACTION_STRETCH,
22        "Stand 2 Legs": PuppyPi.ACTION_STAND_2_LEGS,
23        "Stand 4 Legs": PuppyPi.ACTION_STAND_4_LEGS,
24        "Wave": PuppyPi.ACTION_WAVE,
25    }
26
27    def __init__(self):
28        # acceso al robot
29        self.puppy = None
30
31        # la ventana principal
32        self.root = tk.Tk()
33        self.root.title("Control del Robot PuppyPi")
34        self.root.resizable(False, False)
```



Demo modo texto

Cargue el archivo “**TestPuppyPi.py**” y ejecútelo.



The screenshot shows a code editor with the following content:

```
File Edit Selection ...
PuppyPi

EXPLORER
PUPPYPI
  > __pycache__
  > .venv
  > docs
  > html
  > images
  .gitignore
  GetMessageDetails.py
  LICENSE
  PuppyPi.py
  README.md
  TestPuppyPi.py
  TkPuppyPi.py
  TkPuppyPi.xcf

OUTLINE
TIMELINE

class Test:
    def doActions(self):
        self.puppy.runActionGroup(PuppyPi.ACTION_PEE, pause=4)
        self.puppy.runActionGroup(PuppyPi.ACTION_SIT, pause=4)
        self.puppy.runActionGroup(PuppyPi.ACTION_STAND_4_LEGS, pause=4)
    def doPose(self):
        self.puppy.setPoseLieDown()
        self.puppy.setPose(roll=0, pitch=0, height=70)
    def doMovements(self):
        self.puppy.setPoseStand()
        self.puppy.move(x=5.0, yaw_rate=-5)
        time.sleep(2)
        self.puppy.move_stop()
        time.sleep(1)
    def run(self):
        self.puppy.imageRaw_start(self.processImage)
        self.doActions()
        self.doPose()
        self.doMovements()
        self.puppy.setPoseLieDown()
        self.puppy.imageRaw_stop()
        self.finish()
#
test = Test("192.168.149.1")
test.run()
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13.0 (.venv)